

Forward Pass and Backward Pass

One chain, eight nodes, two rules

Network: $2 \rightarrow 3 \rightarrow 2 \rightarrow 1$ · ReLU · $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$

THE PROBLEM

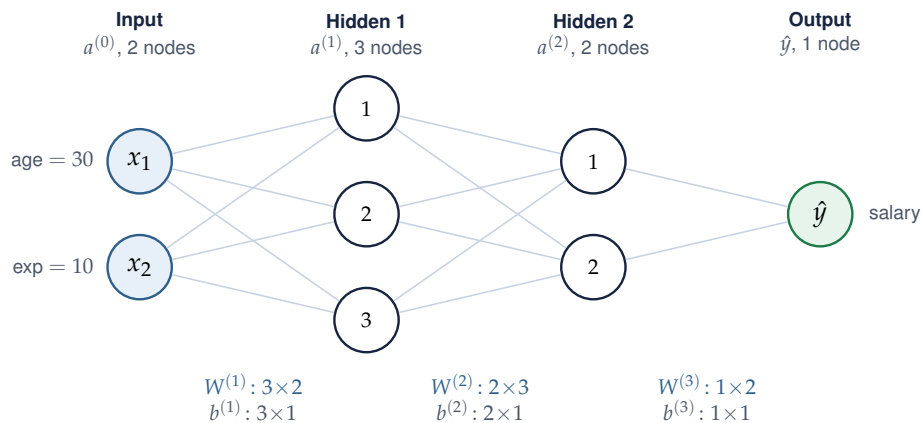
Predict a person's **salary** from two facts about them.

Input 1: age = 30

Input 2: experience = 10

Target: true salary $y = 50$ lakhs

Loss: $L = \frac{1}{2}(\hat{y} - y)^2$



THE NUMBERS WE START WITH

$$W^{(1)} = \begin{bmatrix} 0.1 & 0.3 \\ 0.2 & -0.1 \\ -0.1 & 0.2 \end{bmatrix} \quad b^{(1)} = \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix} \quad W^{(2)} = \begin{bmatrix} 1 & -0.5 & 1 \\ 1 & 0.5 & -1 \end{bmatrix} \quad b^{(2)} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

$$W^{(3)} = \begin{bmatrix} 4 & 2 \end{bmatrix} \quad b^{(3)} = \begin{bmatrix} 2 \end{bmatrix} \quad a^{(0)} = \begin{bmatrix} 30 \\ 10 \end{bmatrix} \quad y = 50$$

$$\text{ReLU}(z) = \max(0, z) \quad \text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$$

Shape rule: $W^{(l)}$ is (nodes in layer l) $\times$ (nodes in layer $l - 1$).

1. The Computational Graph eight nodes in one line



LOOK AT THE ARROWS — THERE ARE ONLY TWO KINDS

Arrow	What it does	Where it appears
Linear arrow	$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$	$a^{(0)} \rightarrow z^{(1)}, a^{(1)} \rightarrow z^{(2)}, a^{(2)} \rightarrow z^{(3)}$
ReLU arrow	$a^{(l)} = \text{ReLU}(z^{(l)})$	$z^{(1)} \rightarrow a^{(1)}, z^{(2)} \rightarrow a^{(2)}, z^{(3)} \rightarrow \hat{y}$

That is the whole network. It alternates: linear, ReLU, linear, ReLU, linear, ReLU — then the loss.

So we only ever need two backward rules. Learn one rule for the linear arrow and one rule for the ReLU arrow, and you can walk back through a network of any depth.

2. Forward Pass walk the chain left to right

STEP F1 $a^{(0)} \rightarrow z^{(1)}$ (linear arrow)

$$z^{(1)} = W^{(1)}a^{(0)} + b^{(1)} = \begin{bmatrix} 0.1 & 0.3 \\ 0.2 & -0.1 \\ -0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 30 \\ 10 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix}$$

$$\text{row 1: } (0.1)(30) + (0.3)(10) + 1 = 3 + 3 + 1 = 7$$

$$\text{row 2: } (0.2)(30) + (-0.1)(10) - 1 = 6 - 1 - 1 = 4$$

$$\text{row 3: } (-0.1)(30) + (0.2)(10) - 1 = -3 + 2 - 1 = -2$$

$$z^{(1)} = \begin{bmatrix} 7 \\ 4 \\ -2 \end{bmatrix} \quad 3 \times 1 \checkmark$$

STEP F2 $z^{(1)} \rightarrow a^{(1)}$ (ReLU arrow)

$$a^{(1)} = \text{ReLU} \left(\begin{bmatrix} 7 \\ 4 \\ -2 \end{bmatrix} \right) = \begin{bmatrix} \max(0, 7) \\ \max(0, 4) \\ \max(0, -2) \end{bmatrix}$$

$$a^{(1)} = \begin{bmatrix} 7 \\ 4 \\ 0 \end{bmatrix}$$

The third one came out negative, so ReLU switched it off. Keep an eye on it.

STEP F3 $a^{(1)} \rightarrow z^{(2)}$ (linear arrow)

$$z^{(2)} = \begin{bmatrix} 1 & -0.5 & 1 \\ 1 & 0.5 & -1 \end{bmatrix} \begin{bmatrix} 7 \\ 4 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

$$\text{row 1: } (1)(7) + (-0.5)(4) + (1)(0) + 1 = 7 - 2 + 0 + 1 = 6$$

$$\text{row 2: } (1)(7) + (0.5)(4) + (-1)(0) - 2 = 7 + 2 - 0 - 2 = 7$$

$$z^{(2)} = \begin{bmatrix} 6 \\ 7 \end{bmatrix} \quad 2 \times 1 \checkmark$$

STEP F4 $z^{(2)} \rightarrow a^{(2)}$ (ReLU arrow)

$$a^{(2)} = \text{ReLU}\left(\begin{bmatrix} 6 \\ 7 \end{bmatrix}\right) = \begin{bmatrix} 6 \\ 7 \end{bmatrix}$$

Both positive, so ReLU changes nothing.

STEP F5 $a^{(2)} \rightarrow z^{(3)}$ (linear arrow)

$$z^{(3)} = \begin{bmatrix} 4 & 2 \end{bmatrix} \begin{bmatrix} 6 \\ 7 \end{bmatrix} + 2 = (4)(6) + (2)(7) + 2 = 24 + 14 + 2$$

$$z^{(3)} = \begin{bmatrix} 40 \end{bmatrix}$$

STEP F6 $z^{(3)} \rightarrow \hat{y}$ (ReLU arrow)

$$\hat{y} = \text{ReLU}(40) = \begin{bmatrix} 40 \end{bmatrix} \quad \text{the network predicts 40 lakhs}$$

STEP F7 $\hat{y} \rightarrow L$ (loss)

$$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(40 - 50)^2 = \frac{1}{2}(-10)^2 = \frac{1}{2}(100)$$

$L = 50$ and the error is $\hat{y} - y = -10$

$a^{(0)}$	$z^{(1)}$	$a^{(1)}$	$z^{(2)}$	$a^{(2)}$	$z^{(3)}$	$\hat{y}$	L
$[30, 10]^T$	$[7, 4, -2]^T$	$[7, 4, 0]^T$	$[6, 7]^T$	$[6, 7]^T$	$[40]$	$[40]$	50

KEEP ALL OF THESE

The backward pass will ask for $a^{(0)}, a^{(1)}, a^{(2)}$ and for $z^{(1)}, z^{(2)}, z^{(3)}$. Write them down now and you will not have to compute anything twice.

3. Backward Pass — The Two Rules one for each kind of arrow

THE IDEA

We want $\partial L / \partial W$ and $\partial L / \partial b$ for all three layers. Instead of writing L as one giant expression, we walk the chain **backwards**, carrying one thing with us: the gradient of L with respect to the node we are standing on.

$$\frac{\partial L}{\partial \hat{y}} \rightarrow \frac{\partial L}{\partial z^{(3)}} \rightarrow \frac{\partial L}{\partial a^{(2)}} \rightarrow \frac{\partial L}{\partial z^{(2)}} \rightarrow \frac{\partial L}{\partial a^{(1)}} \rightarrow \frac{\partial L}{\partial z^{(1)}}$$

Every arrow we cross backwards converts one of these into the next. There are only two kinds of arrow, so there are only two rules.

STEP S Where we start

The last arrow is the loss, $L = \frac{1}{2}(\hat{y} - y)^2$. Differentiate it:

$$\frac{\partial L}{\partial \hat{y}} = \frac{d}{d\hat{y}} \left[\frac{1}{2}(\hat{y} - y)^2 \right] = \hat{y} - y$$

$$\boxed{\frac{\partial L}{\partial \hat{y}} = \hat{y} - y}$$

The $\frac{1}{2}$ is there only to cancel this 2.

STEP A Rule A — crossing a **ReLU** arrow backwards

The arrow was $a = \text{ReLU}(z)$, done element by element. So a_j depends on z_j and on nothing else — one term in the chain rule, no sum:

$$\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} = \frac{\partial L}{\partial a_j} \cdot \text{ReLU}'(z_j)$$

Doing that for every j at once is just an element-by-element multiplication:

$$\boxed{\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \odot \text{ReLU}'(z)}$$

($\odot$ means multiply entry by entry)

$\text{ReLU}'(z)$ is a **switch**: 1 where z was positive, 0 where it was negative. So this rule lets the gradient through where the neuron was awake, and blocks it where it was asleep.

STEP B Rule B — crossing a **linear** arrow backwards

The arrow was $z = Wa + b$. Written out one row at a time:

$$z_j = \sum_k W_{jk} a_k + b_j$$

This arrow gives us three things, because three quantities feed into it.

(i) The gradient of W . The number W_{jk} sits in one place only — inside z_j . One term, no sum:

$$\frac{\partial L}{\partial W_{jk}} = \frac{\partial L}{\partial z_j} \cdot \frac{\partial z_j}{\partial W_{jk}} = \frac{\partial L}{\partial z_j} \cdot a_k$$

Entry (j, k) is (entry j of a column) $\times$ (entry k of a row) — that is an outer product.

(ii) The gradient of b . b_j also sits only inside z_j , with coefficient 1:

$$\frac{\partial L}{\partial b_j} = \frac{\partial L}{\partial z_j} \cdot 1$$

(iii) The gradient of the layer below. Here a_k sits inside *every* z_j , so the chain rule sums:

$$\frac{\partial L}{\partial a_k} = \sum_j \frac{\partial L}{\partial z_j} \cdot \frac{\partial z_j}{\partial a_k} = \sum_j \frac{\partial L}{\partial z_j} W_{jk}$$

That sum runs down a *column* of W , which is a *row* of W^T .

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial z} a^T$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}$$

$$\frac{\partial L}{\partial a} = W^T \frac{\partial L}{\partial z}$$

Forward you multiply by W . Backward you multiply by W^T . Same wires, opposite direction.

ALL FOUR FORMULAS, TOGETHER

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad \frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \odot \text{ReLU}'(z) \quad \frac{\partial L}{\partial a^{(l-1)}} = \left(W^{(l)}\right)^T \frac{\partial L}{\partial z^{(l)}}$$

$$\frac{\partial L}{\partial W^{(l)}} = \frac{\partial L}{\partial z^{(l)}} \left(a^{(l-1)}\right)^T \quad \frac{\partial L}{\partial b^{(l)}} = \frac{\partial L}{\partial z^{(l)}}$$

Shape check that never fails: the gradient of anything has the same shape as that thing. $\partial L / \partial W^{(2)}$ must be 2×3 , so it can only be $(2 \times 1)(1 \times 3)$ — the transpose has just one place to go.

4. Backward Pass — The Numbers walk the same chain right to left

STEP B1 Start at the loss

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 40 - 50 = [-10]$$

STEP B2 Cross $z^{(3)} \rightarrow \hat{y}$ backwards (Rule A)

We stored $z^{(3)} = 40$, which is positive, so the switch is on: $\text{ReLU}'(z^{(3)}) = 1$.

$$\frac{\partial L}{\partial z^{(3)}} = [-10] \odot [1]$$

$$\frac{\partial L}{\partial z^{(3)}} = [-10]$$

STEP B3 Cross $a^{(2)} \rightarrow z^{(3)}$ backwards (Rule B — three results)

We stored $a^{(2)} = \begin{bmatrix} 6 \\ 7 \end{bmatrix}$, and $W^{(3)} = \begin{bmatrix} 4 & 2 \end{bmatrix}$.

$$\frac{\partial L}{\partial W^{(3)}} = \frac{\partial L}{\partial z^{(3)}} (a^{(2)})^T = [-10] \begin{bmatrix} 6 & 7 \end{bmatrix} = \begin{bmatrix} (-10)(6) & (-10)(7) \end{bmatrix}$$

$$\frac{\partial L}{\partial a^{(2)}} = (W^{(3)})^T \frac{\partial L}{\partial z^{(3)}} = \begin{bmatrix} 4 \\ 2 \end{bmatrix} [-10] = \begin{bmatrix} (4)(-10) \\ (2)(-10) \end{bmatrix}$$

$$\frac{\partial L}{\partial W^{(3)}} = \begin{bmatrix} -60 & -70 \end{bmatrix}_{1 \times 2} \quad \frac{\partial L}{\partial b^{(3)}} = [-10] \quad \frac{\partial L}{\partial a^{(2)}} = \begin{bmatrix} -40 \\ -20 \end{bmatrix}_{2 \times 1}$$

The neuron with the bigger weight (4 against 2) gets twice the blame. That is all W^T is doing.

STEP B4 Cross $z^{(2)} \rightarrow a^{(2)}$ backwards (Rule A)

Stored: $z^{(2)} = \begin{bmatrix} 6 \\ 7 \end{bmatrix}$, both positive, so both switches are on.

$$\frac{\partial L}{\partial z^{(2)}} = \begin{bmatrix} -40 \\ -20 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\frac{\partial L}{\partial z^{(2)}} = \begin{bmatrix} -40 \\ -20 \end{bmatrix}$$

STEP B5 Cross $a^{(1)} \rightarrow z^{(2)}$ backwards (Rule B)

Stored: $a^{(1)} = \begin{bmatrix} 7 \\ 4 \\ 0 \end{bmatrix}$, and $W^{(2)} = \begin{bmatrix} 1 & -0.5 & 1 \\ 1 & 0.5 & -1 \end{bmatrix}$.

$$\frac{\partial L}{\partial W^{(2)}} = \begin{bmatrix} -40 \\ -20 \end{bmatrix}_{2 \times 1} \begin{bmatrix} 7 & 4 & 0 \end{bmatrix}_{1 \times 3} = \begin{bmatrix} (-40)(7) & (-40)(4) & (-40)(0) \\ (-20)(7) & (-20)(4) & (-20)(0) \end{bmatrix}$$

$$\frac{\partial L}{\partial a^{(1)}} = \left(W^{(2)}\right)^T \frac{\partial L}{\partial z^{(2)}} = \begin{bmatrix} 1 & 1 \\ -0.5 & 0.5 \\ 1 & -1 \end{bmatrix}_{3 \times 2} \begin{bmatrix} -40 \\ -20 \end{bmatrix}$$

$$\text{row 1: } (1)(-40) + (1)(-20) = -40 - 20 = -60$$

$$\text{row 2: } (-0.5)(-40) + (0.5)(-20) = 20 - 10 = 10$$

$$\text{row 3: } (1)(-40) + (-1)(-20) = -40 + 20 = -20$$

$$\frac{\partial L}{\partial W^{(2)}} = \begin{bmatrix} -280 & -160 & 0 \\ -140 & -80 & 0 \end{bmatrix}_{2 \times 3} \quad \frac{\partial L}{\partial b^{(2)}} = \begin{bmatrix} -40 \\ -20 \end{bmatrix} \quad \frac{\partial L}{\partial a^{(1)}} = \begin{bmatrix} -60 \\ 10 \\ -20 \end{bmatrix}$$

The third column is all zeros, because it multiplied $a_3^{(1)} = 0$. A weight that carried nothing gets no blame.

STEP B6 Cross $z^{(1)} \rightarrow a^{(1)}$ backwards (Rule A — the switch closes)

Stored: $z^{(1)} = \begin{bmatrix} 7 \\ 4 \\ -2 \end{bmatrix}$. The third is negative, so that switch is off.

$$\text{ReLU}'(z^{(1)}) = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \quad \frac{\partial L}{\partial z^{(1)}} = \begin{bmatrix} -60 \\ 10 \\ -20 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

$$\frac{\partial L}{\partial z^{(1)}} = \begin{bmatrix} -60 \\ 10 \\ 0 \end{bmatrix}$$

The -20 was blocked. A neuron that slept on the way forward learns nothing on the way back.

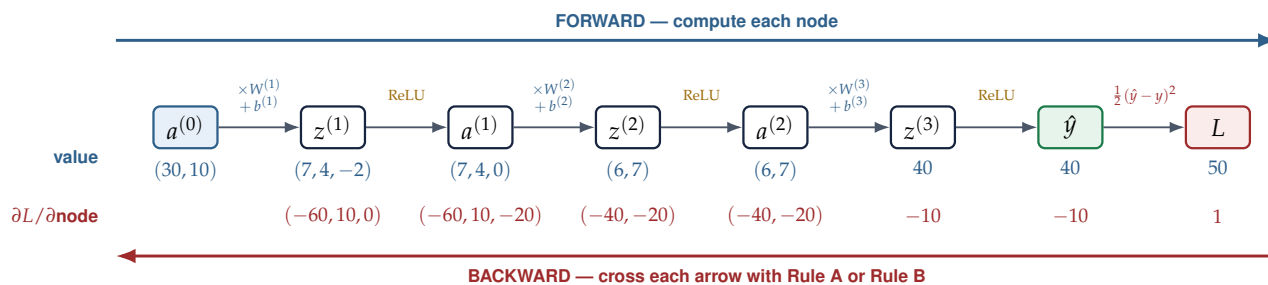
STEP B7 Cross $a^{(0)} \rightarrow z^{(1)}$ backwards (Rule B — last step)

$$\frac{\partial L}{\partial W^{(1)}} = \begin{bmatrix} -60 \\ 10 \\ 0 \end{bmatrix}_{3 \times 1} \begin{bmatrix} 30 & 10 \end{bmatrix}_{1 \times 2} = \begin{bmatrix} (-60)(30) & (-60)(10) \\ (10)(30) & (10)(10) \\ (0)(30) & (0)(10) \end{bmatrix}$$

$$\frac{\partial L}{\partial W^{(1)}} = \begin{bmatrix} -1800 & -600 \\ 300 & 100 \\ 0 & 0 \end{bmatrix}_{3 \times 2} \quad \frac{\partial L}{\partial b^{(1)}} = \begin{bmatrix} -60 \\ 10 \\ 0 \end{bmatrix}$$

The whole third row is zero — the sleeping neuron's weights do not move at all this step.

5. The Whole Thing in One Picture values going right, gradients coming left



READING IT

Blue is what the node computed going right. **Red** is $\partial L / \partial (\text{that node})$ coming left.

Notice the one place the red numbers change shape: at $z^{(1)}$ the -20 became 0 . That is the sleeping ReLU blocking its gradient. Everywhere else the red numbers are just the blue ones' story told backwards.

6. Update and Check do the weights get better?

Layer	$\partial L / \partial z^{(l)}$	$\partial L / \partial W^{(l)} = \partial L / \partial z^{(l)} (a^{(l-1)})^T$	$\partial L / \partial b^{(l)}$
$l = 3$	$\begin{bmatrix} -10 \\ -40 \\ -20 \end{bmatrix}$	$\begin{bmatrix} -60 & -70 \\ -280 & -160 & 0 \\ -140 & -80 & 0 \end{bmatrix}$	$\begin{bmatrix} -10 \\ -40 \\ -20 \end{bmatrix}$
$l = 2$	$\begin{bmatrix} -60 \\ 10 \\ 0 \end{bmatrix}$	$\begin{bmatrix} -1800 & -600 \\ 300 & 100 \\ 0 & 0 \end{bmatrix}$	$\begin{bmatrix} -60 \\ 10 \\ 0 \end{bmatrix}$
$l = 1$			

THE UPDATE RULE

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}} \quad b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}$$

The gradient points uphill on the loss, so we step the other way. Take $\eta = 10^{-5}$.

STEP U New weights

For example $0.1 - 10^{-5}(-1800) = 0.1 + 0.018 = 0.118$.

$$W^{(1)} = \begin{bmatrix} 0.118 & 0.306 \\ 0.197 & -0.101 \\ -0.1 & 0.2 \end{bmatrix} \quad b^{(1)} = \begin{bmatrix} 1.0006 \\ -1.0001 \\ -1 \end{bmatrix} \quad W^{(2)} = \begin{bmatrix} 1.0028 & -0.4984 & 1 \\ 1.0014 & 0.5008 & -1 \end{bmatrix} \quad b^{(2)} = \begin{bmatrix} 1.0004 \\ -1.9998 \end{bmatrix}$$

$$W^{(3)} = \begin{bmatrix} 4.0006 & 2.0007 \end{bmatrix} \quad b^{(3)} = \begin{bmatrix} 2.0001 \end{bmatrix}$$

STEP V Run the chain forward again

$$z^{(1)} : (0.118)(30) + (0.306)(10) + 1.0006 = 3.54 + 3.06 + 1.0006 = 7.6006$$

$$(0.197)(30) + (-0.101)(10) - 1.0001 = 5.91 - 1.01 - 1.0001 = 3.8999$$

$$(-0.1)(30) + (0.2)(10) - 1 = -3 + 2 - 1 = -2$$

$$a^{(1)} = \begin{bmatrix} 7.6006 \\ 3.8999 \\ 0 \end{bmatrix} \quad z^{(2)} = a^{(2)} = \begin{bmatrix} 6.6786 \\ 7.5645 \end{bmatrix} \quad \hat{y} = (4.0006)(6.6786) + (2.0007)(7.5645) + 2.0001$$

$$\hat{y} = 43.8527 \quad L = \frac{1}{2}(43.8527 - 50)^2 = \frac{1}{2}(-6.1473)^2 = 18.89$$

$$\text{Loss: } 50 \rightarrow 18.89 \quad \text{Error: } -10 \rightarrow -6.15$$

Repeat the two passes a few hundred times and the error goes to zero.

THREE THINGS TO REMEMBER

1. **Let the shapes place the transpose.** Every gradient has the shape of the thing it belongs to. $\partial L / \partial W^{(2)}$ must be 2×3 , so it is $(2 \times 1)(1 \times 3)$ — there is no other option.
2. **A zero gradient is not a mistake.** A sleeping ReLU really does have zero gradient, and its weights really do stay put. Too many sleeping at once is the “dying ReLU” problem.
3. **Why η had to be so tiny.** $\partial L / \partial W_{11}^{(1)} = -1800$ while $\partial L / \partial b^{(3)} = -10$, because $\partial L / \partial W^{(1)}$ multiplies by the raw input and age = 30 is large. With $\eta = 10^{-4}$ the prediction overshoots to 79.3 and the loss rises to 429. The real fix is not a smaller η but scaling the inputs before training.

THE WHOLE ALGORITHM

Forward: $a^{(0)} \rightarrow z^{(1)} \rightarrow a^{(1)} \rightarrow z^{(2)} \rightarrow a^{(2)} \rightarrow z^{(3)} \rightarrow \hat{y} \rightarrow L$, storing every node.

Backward: start with $\partial L / \partial \hat{y} = \hat{y} - y$, then walk back across the arrows: **ReLU arrow** $\Rightarrow \partial L / \partial z = \partial L / \partial a \odot \text{ReLU}'(z)$; **linear arrow** $\Rightarrow \partial L / \partial W = \partial L / \partial z a^T$, $\partial L / \partial b = \partial L / \partial z$, $\partial L / \partial a = W^T \partial L / \partial z$.

Update: $\theta \leftarrow \theta - \eta \partial L / \partial \theta$. **Repeat.**

Every number here was computed by hand and checked against a finite-difference test of all 17 gradients (largest disagreement 5×10^{-8}).